Solution Tutorial 5 (core exercises)

1. **Differentiation**
   Let's consider the following function:

$$f(x) = x^2 \sin(x). \tag{1}$$

   (a) Calculate the analytical derivative of this function and plot the function in the range $[0, 2\pi]$.

   (b) Make a function to evaluate the derivative in the same interval using the central difference method, calculated at 200 points evenly spaced. Do this for $h = 1, 0.1, 0.01$ and 0.001. Compare the different h values and compare with the analytical derivative.

   (c) Make a differentiator using Ridder's method to evaluate the same 200 x-values as above, use 3 different target accuracies. Which initial h values yield stable results? Why? Compare the results with part (b).

Solution:

(a) The analytical derivative is $f'(x) = 2x\sin(x) + x^2\cos(x)$ and it is represented by the blue curve in Figure 1.

(b) The solution is presented in Figure 1. To better appreciate the differences, in Figure 2 you can see the relative difference between the analytic and the numeric solutions (i.e., $(f'_{\text{analytic}}(x) - f'_{\text{numeric}}(x))/f'_{\text{analytic}}(x)$), in log scale.

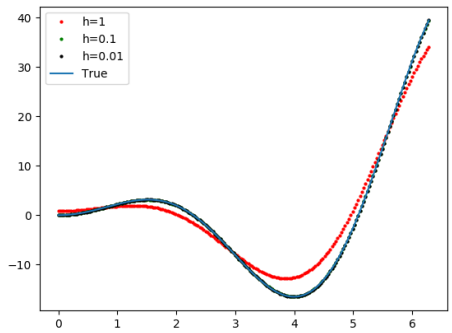


Figure 1: Derivatives of $f(x)$ using the central difference method, compared to the analytical solution (blue line).

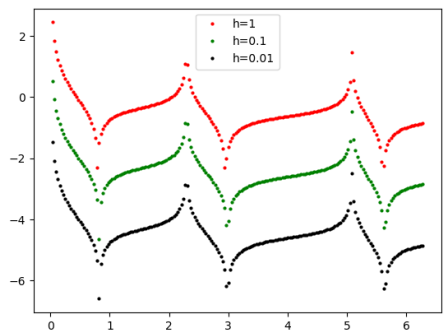
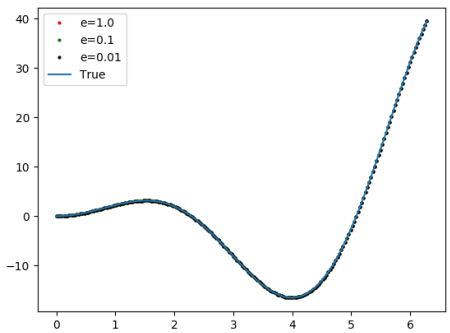
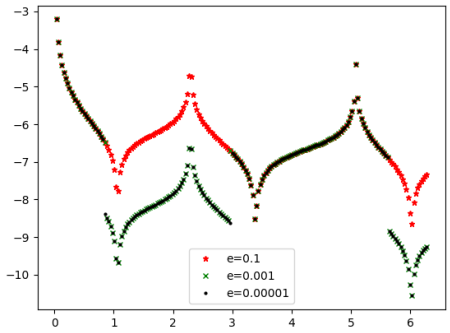


Figure 2: Logarithm of the relative difference between the analytic and the numeric solutions presented in Figure 1.

(c) The solution using Ridder's method is plotted below:



The fractional difference in logarithm is given by:



If the initial h is too small, the first column of $D_{i,j}$ will be dominated by round-off error, propagating numerical noise to the result and breaking stopping conditions.

2. **Generating Random Numbers on a Sphere**

(a) In a spherical coordinate system, any point can be expressed in the units of (r, θ, ϕ). If we consider a unit sphere (i.e. $r = 1$), a random point is equivalent to a random direction characterized by random generation of θ and ϕ. Firstly, generate random points on a unit sphere using the following description of the angles:

$$\theta = \pi P_u(0, 1) \tag{2}$$
$$\phi = 2\pi P_u(0, 1) \tag{3}$$

Here P_u is a uniform random number generator. Use `numpy.random` to generate random values and plot 5000 random points in a 3D plot.

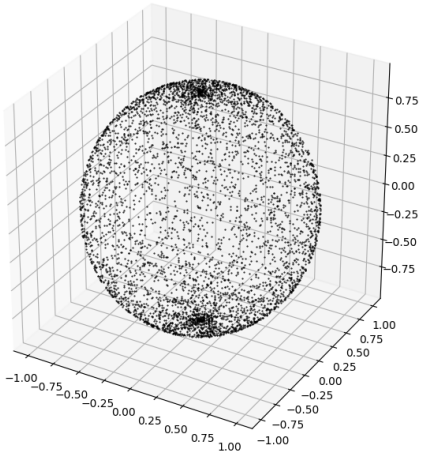
(b) Now use a different definition for randomizing the angles as follows and repeat the above procedure:

$$\theta = \cos^{-1}(1 - 2P_u(0, 1)) \tag{4}$$
$$\phi = 2\pi P_u(0, 1) \tag{5}$$

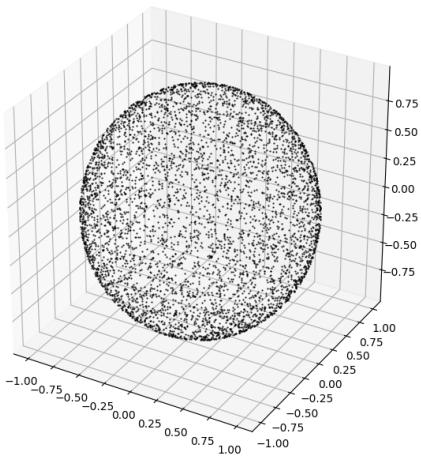
(c) Compare the two plots. Which one is better? Why? Explain how to correctly place random numbers on a sphere and why this holds. Hint: Use the transformation method and remember that the area of each surface element on a unit sphere is $\sin(\theta)\mathrm{d}\theta\mathrm{d}(\phi)$.

Solution:

(a) The first method resulted in an accumulation of points near the poles and a lower density around the equator, as shown in the Figure below. This dishomgeneity happens because on a sphere the area elements becomes smaller when we approach the poles and larger at the equator.



(b) This second method alleviates the issue of lower density of points around the equator:



(c) For the inverse transform method, let's call $p(y)dA$ the probability of having a point in an element of area $dA = \sin\phi d\phi d\theta$. We require this probability to be constant over the sphere, so:

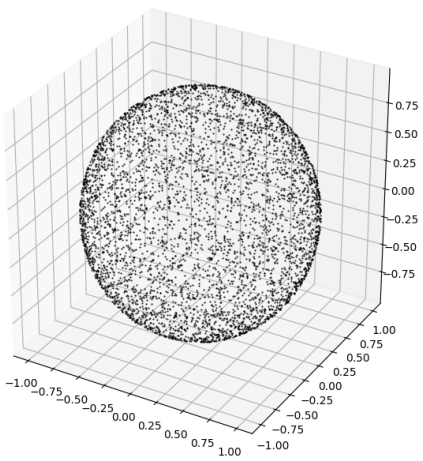
$$p(y)dA \equiv p(\theta, \phi)d\theta d\phi = \frac{\sin\phi}{4\pi} d\phi d\theta \tag{6}$$

This probability function can be factorized into $p(\theta, \phi) = p_1(\theta)p_2(\phi)$. Marginalizing $p(\theta, \phi)$ over ϕ and θ, respectively, we obtain:

$$p_1(\theta) = \frac{1}{2\pi} \tag{7}$$

$$p_2(\phi) = \frac{\sin\phi}{2} \tag{8}$$

$p_1(\theta)$ is already uniformly distributed, therefore it is sufficient to generate uniform random numbers in the interval $[0, 2\pi]$. To generate random numbers distributed as $\sin\phi$ we first compute the C.D.F. $F(\phi)$:

$$F(\phi) = \int_0^{\phi} \frac{\sin\tilde{\phi}}{2} d\tilde{\phi} = \frac{1}{2}(1 - \cos\phi) \tag{9}$$

Applying its inverse $F^{-1}(x) = \arccos(1 - 2x)$ to uniformly distributed random numbers x will generate the required component. The plot will look like: